

Gain Scheduling LQR Algorithm Implemented on Airbrakes for Modern Rocketry Applications

Mason Ma - 13543842

Supervisor Name: Ricardo Aguilera Echeverria

41030 Engineering Capstone - Spring 2024

November 16, 2024

Contents

1	Abstract	3
2	Introduction	4
3	Literature Review	4
4	Rocket Characteristics	5
5	Methodology	5
6	Mechanical Structure of the Airbrakes	6
6.1	Torque Calculation	8
6.2	Gear Calculation	9
7	Avionics	9
8	Control System Architecture	11
9	Velocity Tracking Controller	12
9.1	Equation of motion of the Rocket Model	12
9.1.1	Linearisation Method	13
9.1.2	Gain Scheduling	13
9.1.3	LQR Controller	14
9.1.4	Quadratic Cost Function	15
10	State Estimator	16
11	Drag Coefficient Estimator	16
12	Simulation model	19
13	Simulation results	20
14	Process in the loop validation	24
14.0.1	PIL Results	24
15	Post flight analysis	27
15.0.1	Open loop test	27
15.0.2	Closed loop test	29
16	Findings and discussions	29
17	Conclusion	29
18	Acknowledgments	29
19	Appendices	30

1 Abstract

This paper presents a control algorithm that implements an airbrake system on board a high-powered rocket. Using these actuated airbrakes would allow the rocket to slow down by controlling the longitudinal drag forces and minimise the altitude error that has been set. If successful, this will be flown at AURC (Australian Universities Rocket Competition) in the future, participating in the 10,000 ft category.

2 Introduction

Amateur rocketry is a very popular topic where enthusiasts practise designing, experimenting and building their own model rocket. Different materials could be used to hold the mechanical structure of the rocket like cardboard all the way down to more complex bonds like carbon fibre which strengthen the rocket. To fly a rocket a propulsion system is used which can vary from liquid, hybrid and solid motors. The whole rocket flight typically goes from the rocket staying on pad until the motor lights up entering power ascent where the rocket generates thrust. This is where the force of the thrust overcomes gravity where it will continue burning the propellant increasing altitude until it runs out. This enters the coasting stage where since there is no fuel the only forces that are now acting on it is the weight and drag. This is where just after apogee (the highest peak in flight) the main parachutes will then be deployed allowing for the rocket to have a safer landing speed, as the rocket is going down another parachute which is the drogue will then deploy at a lower altitude ensuring the rocket lands safely with minimal damages. This is the typical stages of a nominal flight in standard amateur rocketry sense. In the UTS rocketry team we are aiming to go to an AURC (Australian University Rocket Competition) competition where many student teams gather their knowledge and insight across different sub sections of the rocket to achieve a desired altitude based on their respective categories. In AURC there are two different categories which is the 10,000ft and 30,000ft competition where the challenge is to get to that altitude as close as possible, we are aiming for the 10,000ft category. A solution to try get to that desired altitude is using a system called the airbrakes. This mechanism is widely used in every different area like trucks, tractors and other commercial vehicles (Subramanian et al., 2006). There are several ways to implement a system to slow down a rocket but normally in modern amateur rocketry applications, airbrakes are a very popular component to control the longitudinal drag force slowing down the rocket at a certain altitude attaining an ideal trajectory. In this paper it is vital to have airbrakes where it provides and modulates the drag of the rocket during the coast stage which is done optimally through a closed feedback loop (Bregin & Khalid, 2024). But there are several challenges of using airbrakes where the motor could overperform or underperform, if there are severe wind conditions or if the velocities at different regions are higher than expected. Which is why “achieving target apogee might not be achievable at any arbitrary state” (Lew et al., 2019). Hence, another solution needs to be addressed, which is why this paper proposes a gain scheduling technique where multiple gains will be implemented for each operating point. That way, the controller will cater to the change in velocity regions of the entire trajectory. The project’s overall aim is to provide a layout for the UTS Rocketry team to use the control system implemented on airbrakes for future competitions. Through getting advice from the supervisor the research question that will be addressed is “Gain Scheduling LQR Algorithm implemented on airbrakes for modern rocketry applications”.

3 Literature Review

In modern industry applications high level systems are implemented, in this research paper it has two main sections which is the guidance control system and the autopiloting technique uses gain scheduling. For industry rockets that use higher impulse propulsion systems, more complex avionic systems are needed for example guidance, INU and autopilot control systems. The guidance system provides an ideal trajectory the rocket should be at based on the user’s input to the system, this will then be transmitted into the autopilot control system. But this could be catastrophic since this ideal trajectory must mimic perfect flight conditions where there isn’t any noise or disturbance. Which is why if something does happen this could lead the rocket to a different direction to the actual setpoint desired which is why this gain scheduling technique implements multiple operating points for the linear model where it uses multiple flight conditions and switches (Atman & Bambang, 2015). Unlike this another research paper utilises a different control system where they use a PID controller, this will then control the airbrakes they have which are in the form of flaps based of a lookup table for their drag profile (Furman et al., n.d.). In comparison the difference between these two papers is that the controller doesn’t use an optimal algorithm where it will change to a different trajectory if there is a disturbance present. Furthermore, an EUCASS paper proposes an improved control algorithm for a student rocket where they implement a chance-constrained algorithm where a lookup table is used with Montecarlo simulations to provide multiple flight profiles (Lew et al., 2019). Many things were considered where they use a probabilistic distribution considering the limits of the airbrakes. Utilising a region of where the rocket is controllable providing a test case for when the airbrakes actuate fully and when retracted based

on different drag profiles. This would allow the user to know a region of where the airbrakes can be used comparing the upper limit where the airbrakes are fully actuated at its max capacity to reach the desired altitude and at the lower limit where airbrakes aren't needed but still reaching the target. Other components have been also considered like the servo response time with respect to actuating the airbrakes, considering and characterising the actuation delay allowing the team to reach within tolerance of 1 percent of 800m. On the other hand, a different approach can be made where this research paper introduces a different method of autopiloting using a pitch axis load factor which will happen during the sustaining phase (Guo et al., 2016). One major problem that control engineers encounter is the thrust properties as the motor doesn't always perform in an ideal situation, this could lead to time varying where the inputs and outputs is not affected with time. Mathematical models have been used to characterise this for the autopilot system but to counteract this problem it could serve as a scheduling parameter. The overall architecture is shown where they use three types of controllers which is a proportional-integral, two proportional controllers and a first order filter (Guo et al., 2016), where their main objective is to have a rocket model with dimensions of roughly 122mm to have travel at a constant altitude during the sustaining phase. Airbrake geometry have been widely considered where it can change the drag profile heavily based on what's chosen, this paper implements an origami shape compared to the pancake model it introduces six times more drag, this can allow the user to have more room for control authority based on the motor used (Lee & Alam, 2021). Overall, several of these published papers introduces the importance of gain scheduling where it incorporates multiple linear models to compute an optimal trajectory for a rocket utilising different operating points. But there are also drawbacks of this technique where it can be very tedious where different gains need to be simulated for each respective operating condition. A more optimal controller could be better implemented to take care of these constraints since incorrectly scheduling offline could lead to bad results. Airbrake shape has also been considered as drag profiles could change drastically based on the geometry. This literature review provided more knowledge on the advantages and disadvantages of my research question.

4 Rocket Characteristics

The rocket is composed of different sub sections where it has systems like recovery, Avionics, propulsion and airbrake system. The airbrakes were placed just around the center of pressure; this would ensure that the airbrakes were stable across the whole duration of the flight, meaning if it were to actuate before intended, it wouldn't cause stability issues where the rocket could be hazard. For the AURC competition it was intended that it would fly on a solid motor which is the M1315 where the target altitude is set for 10000 ft. Open rocket was a software that was used where it allows the user to design and simulate a rocket, through simulating the rocket it can show different characteristics of the rocket flight. For example, once the motor configuration is in and simulated the user can see a six degree of freedom flight simulation with over 50 parameters to analyse graphs like the altitude, velocity and acceleration etc. This would be useful in the following chapters of the paper, where the parameters can then be uploaded into a Simulink file to test the behavior of the rocket and fine-tune the controller to mimic a real flight scenario.

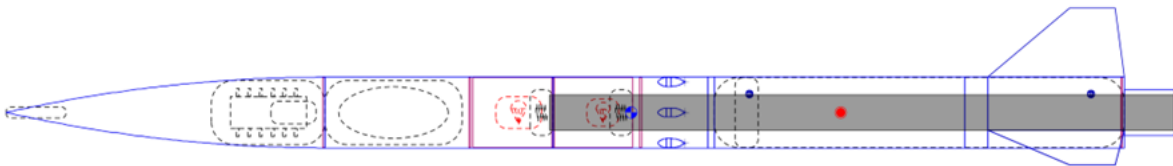


Figure 1: Open Rocket Model of the rocket

5 Methodology

This section will be briefly summarised as the other parts in the report provides a much more detailed explanation of the entire project. So first of all ideation was done in order to find out whats the best way

[illegible]

6 Mechanical Structure of the Airbrakes

6

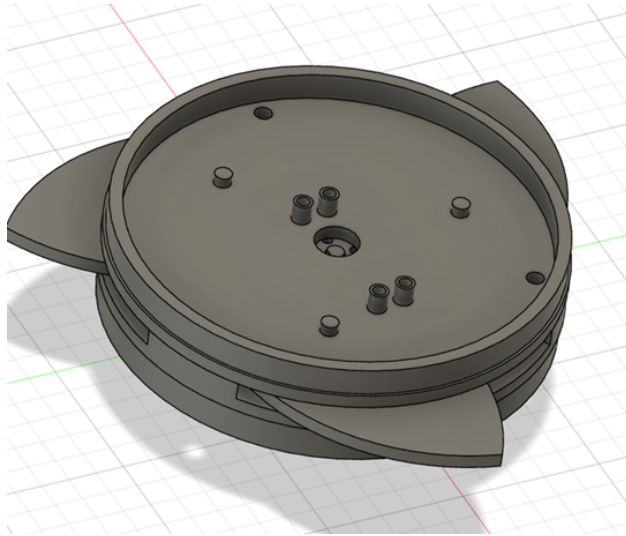


Figure 3: Airbrake system

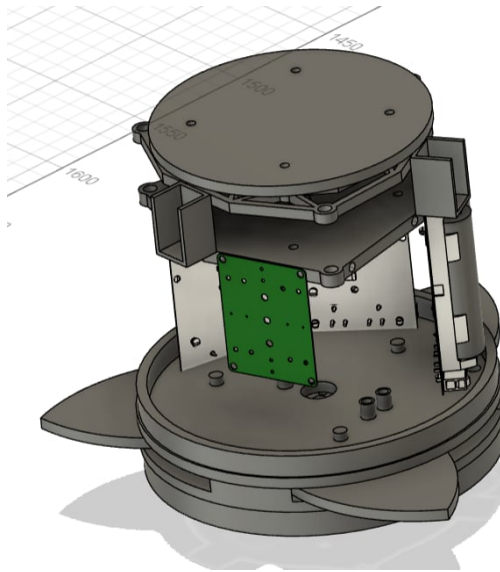


Figure 4: Mechanical Mount of the airbrake system

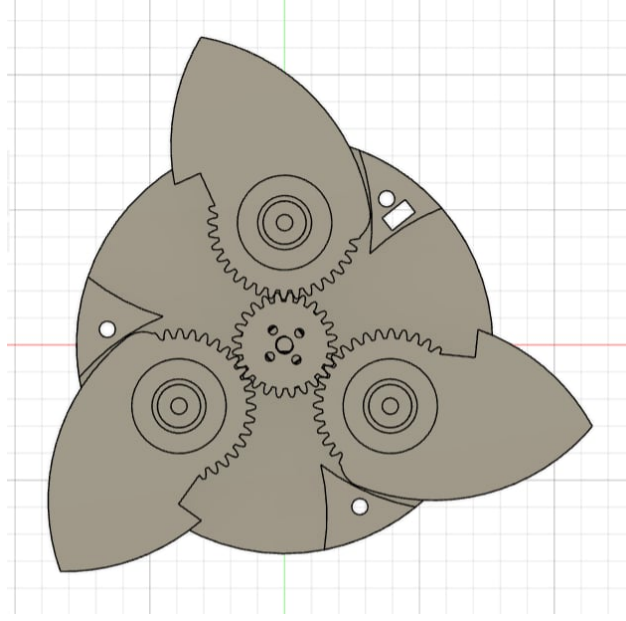


Figure 5: Gear system of airbrakes

6.1 Torque Calculation

In order to determine the servo, the torque needed for the airbrakes extensive calculation are needed to find the most optimal component for the ranges of forces that are experienced. Based off the mechanical team it was found out that the mass of the paddle is 55.17 g, the mass of the bolt is 7.85 g and the surface area of the airbrakes is 2057.693 mm². Given this information the torque can then be calculated finding out how much force it will need for it to actuate the paddles out. The torque needed to overcome the aerodynamic load can be shown below where it is assumed that it has a drag coefficient of a triangle which is roughly around 1.17, velocity of 274 m/s and air density of 1.225 kg/m³. The airbrake paddle area can also be calculated which is roughly around 0.002057693 m².

Dynamic load calculated as followed:

$$F = \frac{1}{2} \times 1.225 \times 274^2 \times 1.17 \times 0.002057693 = 110.7066376220805 \text{ N}$$

or

$$F = \frac{1}{2} \rho v^2 C_d A = 110.7066376220805 \text{ N}$$

The torque to overcome the dynamic load can then be calculated using the formula below where r is the radius of the paddle.

$$\tau_{\text{aero}} = Fr = 110.7066376220805 \times 0.057290000000000001 = 6.34238326937 \text{ Nm}$$

The torque of each bearing also needs to be calculated to examine how much force it will generate once the airbrakes actuate which would create a resistance force back into the gear system.

$$F = \frac{M}{d} = \frac{6.34238326937 \text{ Nm}}{0.01} = 634.238326937 \text{ N}$$

Where d is the distance between the centerline of the bearing to the airbrake itself. (Times it by 2 as there are two bearings).

F becomes the normal reaction force for the bearing.

$$\tau_{\text{bearing}} = F \times \mu = 634.238326937 \times 0.0015 = 0.9513574904$$

6.2 Gear Calculation

Gear calculations are crucial as it is the system that will move the airbrakes and provide the control precision and effectiveness under high velocities and drag. As shown above that the torque was calculated to ensure that the servo component implemented is enough force to overcome the aerodynamic load when the airbrakes are actuated. Gear teeth ratios allow you to control the movement of the airbrakes with respect to the force of the servo through different increments. This would allow the user to have fine control on how the airbrakes move optimally, the gear ratios also have an impact on the speed of the airbrakes where it offers precise actuation speeds where the correct ratios prevent overshooting or undershooting the positions it should be while not actuating too fast or slow. These gear ratios will also be carefully considered under different loads making sure that the forces spread evenly maintaining and maximising the efficiency of the system.

As the team found the torque that calculated was easily able to control the motor to the desired speed the gear ratios wasn't needed as much but was found to be roughly 1:1.5.

The gears were also tested and calculated for its bending, load transmitted and velocity factor to ensure that it will be able to tolerate such forces on the rocket preventing any uncontrollable behaviours.

7 Avionics

Avionics play a critical role, serving as a communication device between the microcontroller controlling the servo, which commands a surface area to be exposed for the airbrake system. It allows the controller to make a correct decision on when to stop actuating the airbrakes based on the sensors on the avionics board, determining the rocket's current state of flight. The components on the board include the IMU, which provides useful data on the rocket's acceleration and orientation, and a barometer, which measures the rocket's pressure. These sensors provide feedback on the current state of the rocket, which is fed into the controller to decide when to slow down the rocket.

Component Selection:

The components were carefully chosen using a morph table to evaluate the pros and cons of each sensor. The final sensors chosen are listed below as best suited for our system and airbrake requirements.

Barometer: BMP581

This sensor was selected for its increased accuracy in pressure measurement, essential for correct altitude estimations. It provides extensive documentation on various communication lines like SPI or I2C, as well as usage examples. Its small size allows more flexibility with the PCB constraints while considering parameters such as operating voltage and current consumption.

IMU: BNO085

Due to the size constraint onboard the rocket, components like the gyroscope, accelerometer, and magnetometer are essential for improving altitude estimation through various sensor fusion algorithms. As a 9-degree-of-freedom component in a single sensor, it significantly reduces the required components. This sensor also provides ample documentation for communication lines like SPI or I2C, along with usage examples. It also supports an external crystal, which helps to reduce sensor noise and improve clock timing.

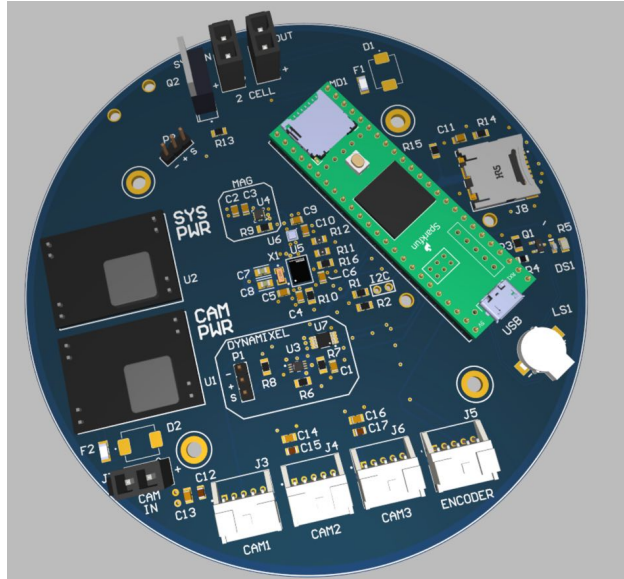


Figure 6: 3D view of CAS Board

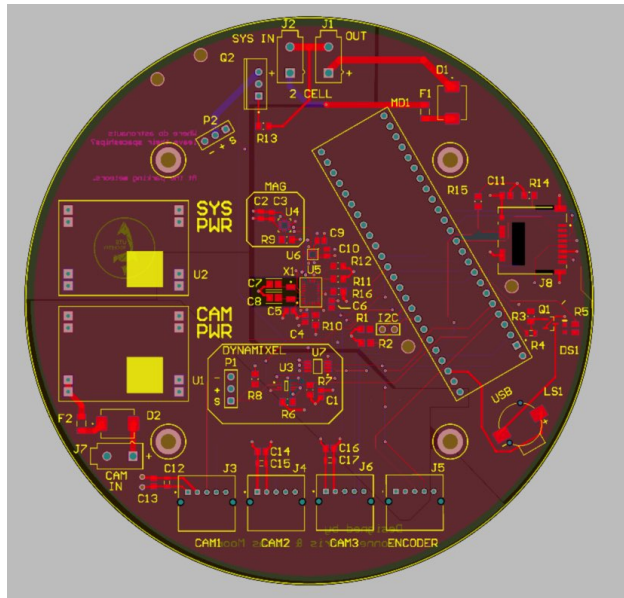


Figure 7: Traces of CAS Board

8 Control System Architecture

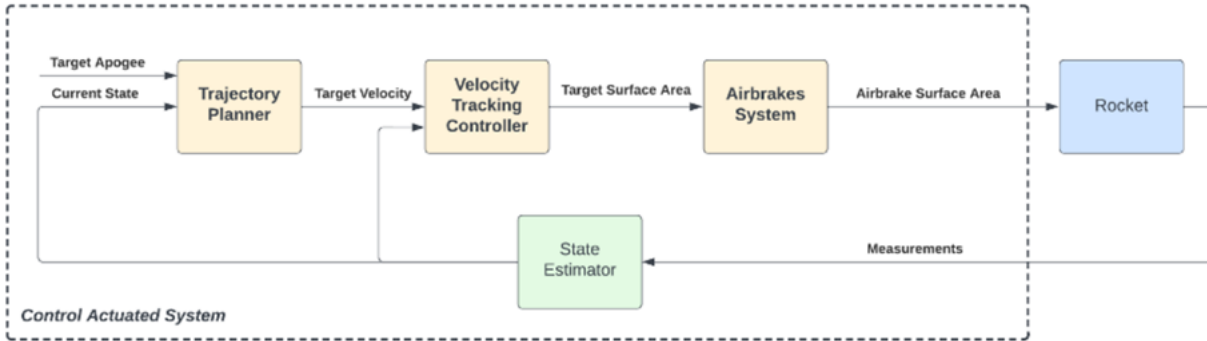


Figure 8: Overall Architecture of the Control System

This Control system flowchart has several components connected to each other in the loop, and this is vital as it will allow the rocket to reach the desired apogee by using airbrakes to control the drag. This will allow the controller to make optimal decisions during the coasting stage where it will actuate based on the desired velocity it reaches every time sample to reach the desired setpoint. The controller is designed in MATLAB and Simulink, where it will then be tested in SIL and then implemented in the microcontroller, where PIL will be used to validate between the simulation and hardware. This will allow various flight scenarios to be mimicked to test the validity of the controller, making sure there are no software bugs on the day before the actual flight tests. This control diagram consists of several key components, which are the Trajectory planner, Velocity tracking controller, Airbrake system and State estimator (further detailed explanation will be done in the later chapters as this just provides a summary).

Trajectory Planner This block predicts the next instantaneous desired velocity to reach apogee. It uses a predictive equation which includes parameters like velocity, air density and drag to compute its next desired velocity. This equation can then be reformulated to find the desired distance where it will show the desired velocity needed to get to that distance at that time instance.

Velocity Tracking Controller Based on the desired velocity it will then be fed into the controller where its main objective is to converge the error signal to zero comparing both the rocket's current velocity measured from sensors to the desired velocity computed from the trajectory planner. This is effectively a 2-stage controller where it uses parameters taken in from the trajectory planner to take into account of the dynamics of the rocket where the controller provide a precomputed gain allowing the airbrakes to provide a surface area to be exposed at that instant.

State Estimation Based on the system it will require an observer where the states are able to be measured and fed back into the control loop. Allowing the controller to make decisions on the information of where the rocket is and what the desired velocity to reach apogee through exposing the airbrakes. The sensors onboard will then measure at each time step where it will use a sensor fusion algorithm to filter the data allowing for accurate measurements.

Airbrake System The airbrake system uses a gear orientated system which will have 3 airbrake paddles spread uniformly in a shape of a leaf design. A servo will also be used to move the airbrakes but since it is servo angles the surface area exposed will need to be converted for the servo to know what PWM signal will reflect the area exposed. A calibrated equation will be implemented where it will describe a relationship with the required servo shaft angle needed to extend the airbrakes at that surface area.

9 Velocity Tracking Controller

The aim of this controller is to minimise the error between the rocket's current velocity and required velocity based on the trajectory planner. This would allow it to output an optimal surface area for the airbrake to be exposed out of the rocket.

9.1 Equation of motion of the Rocket Model

The forces on the rocket need to be understood first to achieve a nonlinear equation of motion of the rocket. The forces acting upon the rocket could be treated at the basic level where there will be the rocket's weight, atmospheric drag of the rocket including airbrakes and its thrust force. This equation can be represented as follows:

$$F_{\text{net}} = F_{\text{thrust}} - F_{\text{drag}} - F_G$$

Where:

- Thrust force of the rocket:

$$F_{\text{thrust}} = T$$

- Drag force:

$$F_{\text{drag}} = \frac{1}{2} \rho V^2 C_d A$$

where:

- ρ = density of fluid
- V = speed of object relative to the fluid
- C_d = drag coefficient
- A = cross-sectional area

- Gravitational force:

$$F_G = mg$$

So, the full equation becomes:

$$F_{\text{net}} = T - \frac{1}{2} \rho V^2 C_d A - mg$$

This equation can then be expanded where there will be two drag forces: one for the rocket itself and the other is for when drag is also imposed on the airbrakes when it is actuated out.

$$F_{\text{net}} = F_{\text{thrust}} - F_G - F_{\text{DR+Airbrakes}}$$

$$ma = T - mg - \frac{1}{2} \rho V^2 C_{DR} A - \frac{1}{2} \rho V^2 C_d A$$

Both drag force equations are coupled together. Since the exposed surface area we are trying to control is the drag force of airbrakes, the parameter is then changed to “ U ” as that will be represented as the control input.

$$ma = T - mg - \frac{1}{2} \rho V^2 C_d A - \frac{1}{2} \rho V^2 C_d U$$

Hence, with this newly acquired nonlinear model it can be represented as such:

$$m\ddot{x} = T - mg - \frac{1}{2} \rho \dot{x}^2 C_d A - \frac{1}{2} \rho \dot{x}^2 C_d U$$

$$m\ddot{x} = T - mg - \frac{1}{2} \rho \dot{x}^2 C_d (A + U)$$

9.1.1 Linearisation Method

Based on the equation of motion, we can apply linearisation techniques which can help ease the analysis and build a controller based on the model. Since this is a linear representation, the accuracy will eventually decrease as you go further from the linearisation point that's chosen. As the rocket's trajectory changes over time, linearisation at one operating point would be very inaccurate. For example, if you linearise around $v = 0$, but if it's after motor burnout, your system is very inaccurate as the error will build up as it will eventually be very far from that linearisation point. Hence, the proposed idea is to linearise across the operating point $(v_{\text{ref}}, u_{\text{ref}})$. Based on the trajectory planner, it will provide a reference velocity that is constantly changing; to account for that, it can be linearised across that point to consider the constant change in velocity.

Taylor Approximation is a method that will be used to linearise the nonlinear model across a certain linearisation point as follows:

$$m\ddot{x} = T - mg - \frac{1}{2}px^2C_d(A + u)$$

Let $\dot{x} = v$ and $x = v$.

$$m\dot{v} = T - mg - \frac{1}{2}pv^2C_d(A + u)$$

$$\dot{v} = \frac{T}{m} - g - \frac{1}{2m}pv^2C_d(A + u) = f(v, u)$$

Linearisation at $(v_{\text{ref}}, u_{\text{ref}})$:

$$f(v, u) \approx f(v_{\text{ref}}, u_{\text{ref}}) + \frac{\partial f}{\partial x}(v_{\text{ref}}, u_{\text{ref}})(v(t) - v_{\text{ref}}) + \frac{\partial f}{\partial u}(v_{\text{ref}}, u_{\text{ref}})(u(t) - u_{\text{ref}})$$

$$f(v_{\text{ref}}, u_{\text{ref}}) = \frac{T}{m} - g - \frac{1}{2m}pv_{\text{ref}}^2C_d(A + u_{\text{ref}})$$

$$\frac{\partial f}{\partial x}(v_{\text{ref}}, u_{\text{ref}})(v(t) - v_{\text{ref}}) = \left(-\frac{1}{m}pv_{\text{ref}}C_d(A + u_{\text{ref}}) \right) (v(t) - v_{\text{ref}})$$

$$\frac{\partial f}{\partial u}(v_{\text{ref}}, u_{\text{ref}})(u(t) - u_{\text{ref}}) = \left(-\frac{1}{2m}pv_{\text{ref}}^2C_d \right) (u(t) - u_{\text{ref}})$$

$$f(v, u) \approx \left(-\frac{1}{m}pv_{\text{ref}}C_d(A + u_{\text{ref}}) \right) (v(t) - v_{\text{ref}}) + \left(-\frac{1}{2m}pv_{\text{ref}}^2C_d \right) (u(t) - u_{\text{ref}}) + \frac{T}{m} - g - \frac{1}{2m}pv_{\text{ref}}^2C_d(A + u_{\text{ref}})$$

$$f(v, u) \approx a\Delta v + b\Delta u + \delta$$

9.1.2 Gain Scheduling

Gain Scheduling is an important technique in modern control theory, this algorithm is mainly for nonlinear systems where the dynamical model of the system changes based on different operating points. This is implemented since one controller isn't sufficient as it would eventually sway from the operating point, or the desired performance needed for the multiple operating ranges inside the plant model. This is why it is an important technique used onboard a rocket where the conditions would change each operating condition. This is because during the whole trajectory, the parameters and conditions change, such as the velocity and density. The controller will then respond differently based on these conditions, where only one won't be enough across the whole flight as it will sway from the desired operating point, leading to over-saturation leading to uncontrollability. This is why a family of linear controllers would be vital, as it would satisfy conditions based on higher velocities and lower velocities.

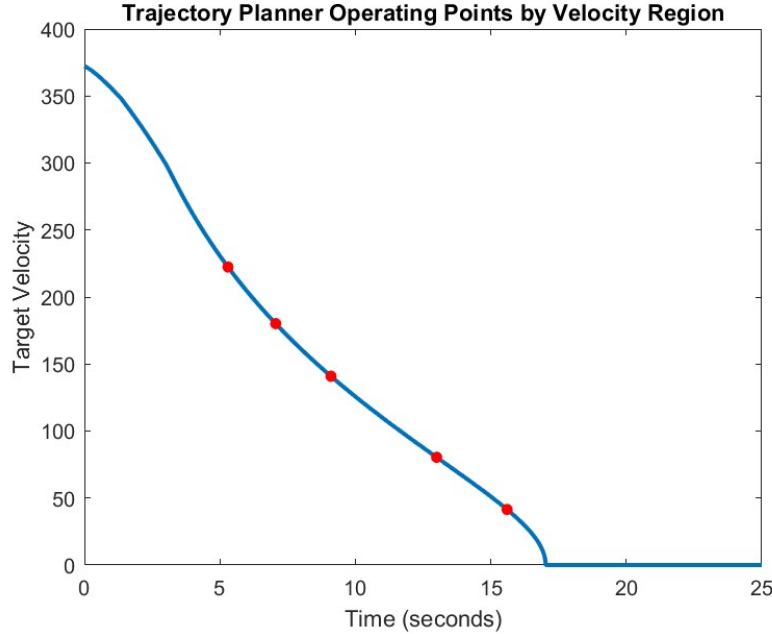


Figure 9: Gain Scheduling for different operating points

9.1.3 LQR Controller

The linear quadratic regulator is an optimal control algorithm which provides an ideal gain based on the cost function. Where minimising this cost function allows the user to balance the system states and inputs. The key difference and the reason LQR were chosen rather than the other controllers like PID and MPC is because of its simplicity as it takes into the account of the dynamical model while considering of different variables as it can be designed to handle multiple inputs and outputs (MIMO). The reason PID wasn't chosen was since it can't handle several inputs/parameters, it also doesn't have an optimisable cost function which can lead to poor performance. LQR also uses a state space model allowing it to consider of the full state of the system, while PID doesn't really account for the state of the system but rather the error signals between desired and actual measured points. On the other hand, Model predictive control was considered as it uses the algorithms of LQR, it can also handle more complex dynamics and able to manage multiple constraints upon the system. But this is for more advanced applications and implementing it on a microcontroller can be computationally expensive which is why it wasn't considered as it impacted the loop speed.

The linear quadratic regulator is an optimal control algorithm which provides an ideal gain based on the cost function. Where minimising this cost function allows the user to balance the system states and inputs. The key difference and the reason LQR were chosen rather than the other controllers like PID and MPC is because of its simplicity as it takes into the account of the dynamical model while considering of different variables as it can be designed to handle multiple inputs and outputs (MIMO). The reason PID wasn't chosen was since it can't handle several inputs/parameters, it also doesn't have an optimisable cost function which can lead to poor performance. LQR also uses a state space model allowing it to consider of the full state of the system, while PID doesn't really account for the state of the system but rather the error signals between desired and actual measured points. On the other hand, Model predictive control was considered as it uses the algorithms of LQR, it can also handle more complex dynamics and able to manage multiple constraints upon the system. But this is for more advanced applications and implementing it on a microcontroller can be computationally expensive which is why it wasn't considered as it impacted the loop speed.

9.1.4 Quadratic Cost Function

The quadratic cost function is a mathematical expression that is used to balance the overall system performance through either penalizing the system states or the overall control effort that's acted upon the system.

$$J = \int_0^\infty (x(t)^T Q x(t) + u(t)^T R u(t)) dt \quad (1)$$

This is done through balancing the Q and R matrices where Q gives you information of on how to weigh each of your desired states, this allows you to consider which states are critical. While the R matrices provide the penalisation of the control effort where you can adjust how aggressive or slow the system response should be in the actuator.

Since it is a single-state system, the Q and R matrices can be tuned intuitively where the velocity state will have high weight as there is less concern about the changes in states of the system. As Gain scheduling is also used, it will also counteract if the controller does sway from its desired operating point. The R-value is also limited so that the airbrakes are not overly aggressive while also preventing any sort of saturation effect.

Due to the nature of the rocket constantly changing its trajectory throughout its flight, a single optimal gain wouldn't be efficient enough to reach the desired altitude needed using the airbrake. This is why multiple linearisation points will be taken for v_{ref} by using the trajectory planner.

Thus, the controller will be constantly updated through the changes in the A and B matrices.

Note: Due to competition rules the airbrakes are only allowed to be used after motor burnout which is why the points picked will be after this case.

The linearisation points ($v_{\text{ref}}, u_{\text{ref}}$) were picked carefully through finding the linearisation points needed (shown in Figure 4), where this will change what u_{ref} is needed. Through steady state analysis, u_{ref} was derived as follows:

$$u_{\text{ss}} = u_{\text{ref}} = -A + \frac{-2gm}{\rho \times v_{\text{ref}}^2 \times C_d}$$

After obtaining the linear model and its matrices, a controller was then derived in MATLAB, where it is just a single state system:

$$\begin{aligned} A_c &= \left(\frac{-1}{m} p v_{\text{ref}} C_d (A + u_{\text{ref}}) \right) \\ B_C &= \left(\frac{-1}{2m} p v_{\text{ref}}^2 C_d \right) \\ C_C &= 1 \\ D_C &= 0 \end{aligned}$$

The gains were then derived as shown below, where they will then be fed into the controller equation in a MATLAB function block:

$$u_k = -K(x_k - x_{\text{ref}k}) + u_{\text{ref}k}$$

Where:

x_k = rocket velocity

$x_{\text{ref}k}$ = reference velocity

10 State Estimator

Sensor Fusion is an algorithm that combines multiple sensors together in order to reduce the error deviation in the system. This would enable reliability in the trust of information being processed. In terms of implementing in a system onboard a rocket, it is vital that sensor fusion and filtering is incorporated as this would allow proper estimates on the altitude, velocity, acceleration and orientation etc. In terms of the rocket, the avionics that connect and interface with the controller is the CAS board where it has components like the IMU and Barometer that measure the current state of the rocket. Implementing sensor fusion would allow for the integration of the IMU which measures the acceleration and orientation, with the data in the barometer which measures pressure. Through fusing these readings together would ensure reliability and provide the user more accurate information of the rocket's current state.

Originally a 1DOF Kalman filter was incorporated where it was purely using the barometer to filter and measure the future states of the rocket. From our first open loop test launch to verify the impact of airbrakes compared with simulation when the airbrakes are actuated during coasting stage. Upon the airbrakes being actuated, this created a turbulent flow which could lead to a pressure variation across the rocket. Meaning these disturbances could lead to incorrect data being measured by the barometer which could have spread false information on how much of an impact the airbrakes were during the flight. Which is why in order to alleviate that problem for the next flight, sensor fusion was utilised where instead of using a linear model of the rocket, it was decided that an extended Kalman filter will be used where a nonlinear model would be incorporated to represent accurate dynamics of the rocket. This would be coupled with the IMU fused together where if the airbrakes actuated more trust would be put on that sensor measuring the acceleration of the rocket rather than the pressure from the barometer.

11 Drag Coefficient Estimator

Drag plays a pivotal role in the controller architecture as it directly influences the drag force which the airbrakes will generate when actuated. This will allow the controller to make a correct decision on when to actuate the airbrakes to reduce the velocity to a desired altitude. The controller is evidently a two-stage procedure where it first goes to the trajectory planner where one of the inputs is drag coefficient, it then outputs a desired velocity at that instance which will then be fed into the controller to decide on the control surface needed to be exposed at that instance to slow down the rocket to its desired height. To achieve this drag coefficient will need to be predicted at every time instant, but due to it being computationally expensive for the microcontroller to run, an offline CD estimator will be used. To achieve this based on the predictive equation that the trajectory planner uses (see section for more info?) it will require an accurate prediction of CD, and this input will change based on different planned velocities. ANSYS was a tool that was used to conduct CFD simulations for various velocities during the coasting stage. In summary was done through having a rocket model with airbrakes uploaded to the simulation file where a CAD geometry of an enclosure was first implemented for the air. The enclosure will then be meshed which means dividing it into lots of small cells that individually calculate the airflow moving through them. This will then allow the user to set a boundary condition which means all the parameters like velocity, density and surface area to solve the simulation. Through running the simulation, it will then output results like contour or pathlines and providing useful data on what the drag coefficient plot would look like at different airbrake extensions which is the key to making a CD estimator. Running this simulation over again for different results will provide a relationship relating fin extension to drag coefficient for each set of velocity experienced on the rocket. Through getting different data sets a second order polynomial equation was then implemented where it would act as a scheduling algorithm for when the rocket enters a specific velocity region it will output a desired CD at that time instance of velocity and airbrake extension.

A 2nd order polynomial equation was used to replicate a switching algorithm were based on the different airbrake extensions and velocity regions it will provide a drag coefficient. As it's a simpler model this would be computationally fast and efficient since it finds the linear relationships between each data set of

inputs and outputs like velocity, density and area with drag coefficient. The images below show the accuracy of this prediction model as it does follow the trends of the raw data, but there are complications where it is not fully accurate as that's due to the nature of the model, where it is a linear approximation. Ultimately this model chosen over the other models as it provides less computations for the microcontroller while maintaining some accuracy over the predictions.

$$Cd = c_0 + c_1A + c_2\rho + c_3v + c_4A^2 + c_5A\rho + c_6Av + c_7\rho^2 + c_8v^2$$

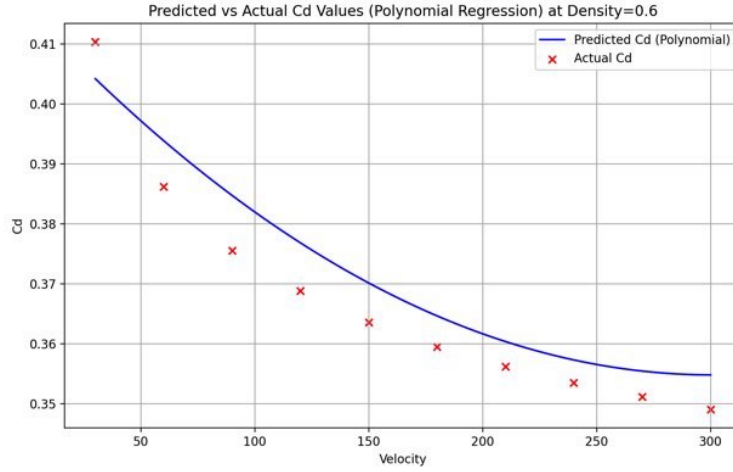


Figure 10: Linear regression using a 2nd order polynomial for Cd of 0.6

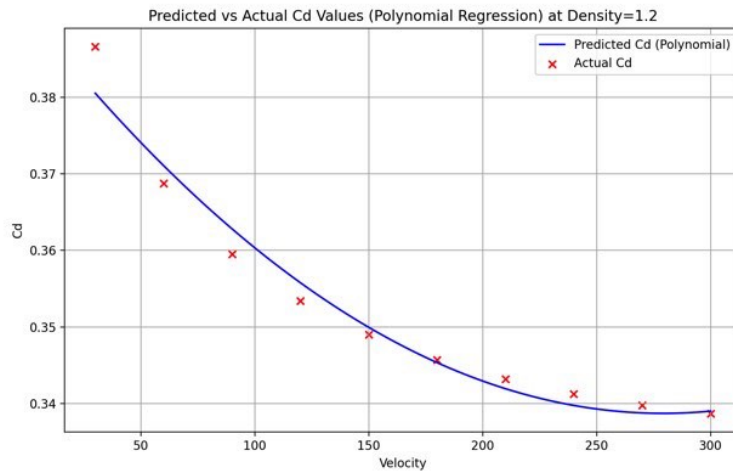


Figure 11: Linear regression using a 2nd order polynomial for Cd of 0.9

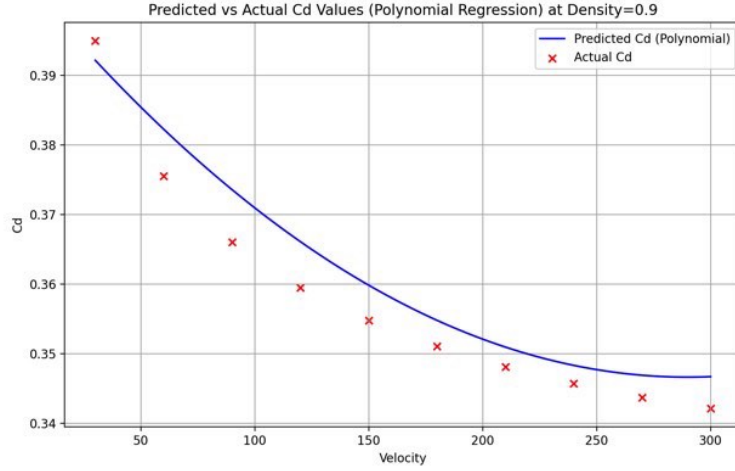


Figure 12: Linear regression using a 2nd order polynomial for Cd of 1.2

A gaussian process regression model was also a method used where it consists of a probabilistic model which can characterize any non-linearity well, especially with drag coefficient in play. It does this through considering some scenarios where it defines and distributes the function based on the data that's been given. So, if more data is given the better the model will be trained on predictions and any uncertainties with the set given. For estimating the drag coefficients, the GPR model will take the input parameters like area, velocity and density to output an ideal Cd, where it will provide a confidence interval to see how accurate it thinks the prediction is. The problem with going ahead with this model is that it is computationally expensive, severely slowing down the microcontroller while maintaining other operations in the background. Especially with large data sets and having a high number of coefficients in the predicted model.

Another method that could have been used Spline interpolation where it implements a piecewise function to connect the data points together, this in turn smooths the data creating a line through each set providing a smooth curve. The image that's shown below is a 4-degree 4-knot spline function, and the number of coefficients greatly impacts the model's performance. As this model uses a high degree in order to provide a smooth prediction it was decided that this couldn't be used since it was too complex and would be computationally heavy for the microcontroller.

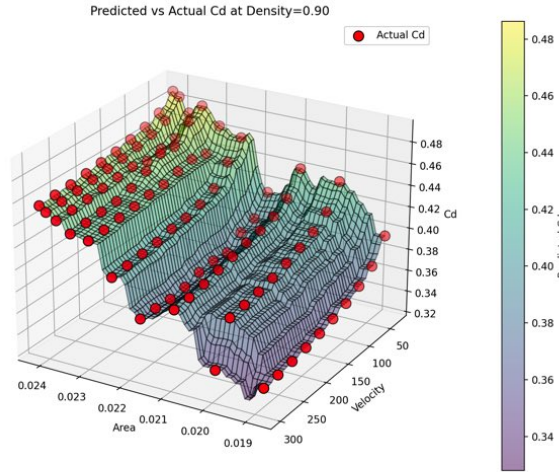


Figure 13: Spline Interpolation for Cd of 0.9

12 Simulation model

Drag Coefficient Estimator This estimator uses a 2nd order polynomial equation which as inputs of total reference area, density and velocity. This will then output a drag coefficient at that current velocity which can then be fed into the trajectory planner.

Trajectory Planner This provides the next instantaneous velocity required through parameters inputted like target apogee, current velocity, total rocket area, density and drag coefficient at that time instant. This will provide a desired velocity needed to reach apogee at that point, the error signal between target apogee and current velocity will decrease and this will in turn mean the desired velocity will start to decrease until it converges to 0 meaning the desired altitude will then be hit.

Velocity Tracking Controller This is a linear quadratic regulator controller where the gains are simulated offline, this performs gain scheduling for different operating points meaning it will take account for higher and lower velocity regions. This takes in the desired velocity from the trajectory planner and the current velocity of the rocket which will then have an error signal multiplied by the gain. This would output an optimal gain which will feed into the airbrake system block.

Apogee Prediction Model Apogee prediction model is also implemented where it uses the equations of the velocity tracking controller but rearranged. This means based on the velocity ahead of time it would provide the steady state altitude.

Airbrake System The airbrake system block uses a nonlinear second order actuator which will then use the output of the controller to limit the area between zero airbrake position to max airbrake area. The rate limit determines how fast the airbrake goes in and out, the output will then give a surface area exposed by the airbrakes at that time instance.

Area to Angle Conversion In terms of implementing it on the actual rocket, the surface area that is exposed is controlled by the servo's PWM signal. Therefore, the area needs to be converted into a servo angle for it to understand how much to actuate the airbrakes. This is where shown below is the mathematical representation of the relationship between the airbrake area and servo angle, allowing us full control for when a given commanded surface area is needed. This equation maps out the given angle based on that.

$$\text{angle} = \frac{-b + \sqrt{b^2 - 4a(k_{\text{MinArea}} - \text{area})}}{2a}$$

Non linear rocket model The output of the airbrake system will then be fed into the input of the rocket model where it adds the rocket reference area to account for the total rocket area. The rocket model is using the equation of motion to mimic the rocket flight path on the day where it includes forces like drag, thrust and gravity. The total rocket area will then be fed into the drag force block where it will use the equation of motion to provide outputs of acceleration, velocity and height using the integrator block. Parameters like height, velocity, total rocket area and drag coefficient will then be fed back into the blocks above, where the loop continues until the desired apogee is reached. Density is also calculated and fed back into the CD estimator using the ISA atmosphere block which provides the current density based on the altitude from the rocket model.

13 Simulation results

Closed loop Simulations were then done to test the behaviour of the system with the controller where a target altitude was set. This will allow us to see the airbrake being exposed and visualise if it slows down to that desired altitude or not.

The target setpoint was 2700m: In the trajectory planner graph, the airbrakes will actuate after 3 seconds (this is when the rocket is coasting allowing for control decisions to happen), after 3 seconds the velocity curve matches the trajectory planner meaning the error signal between them converges to 0. This would mean that the velocity tracking controller is matching the desired velocity each time for when the airbrakes are actuating.

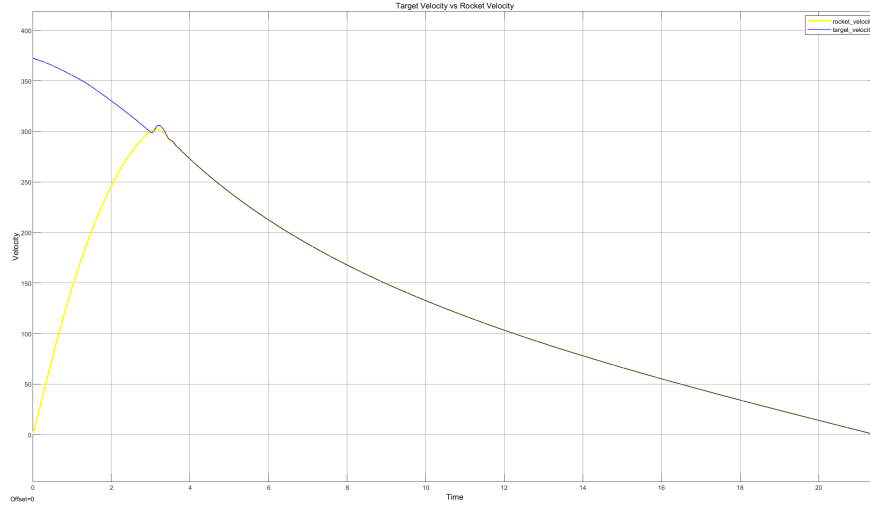


Figure 14: Trajectory planner targeting 2700m

The airbrake extension can be seen here where it will track the velocity curve, it actuates a bit under half extension in the first half of the flight deciding that if the controller actuates more than this under high velocities and drag it might undershoot. This is where the gain scheduling algorithm comes into play where if it starts drifting from its operating condition then it will swap to a different gain based on the velocity region that's met. In the second half of the graph, the airbrakes are gradually increasing in its exposed surface area where it will actuate almost to max extension (no saturation occurs). The airbrake area will then go back to zero position gradually meaning the desired altitude has been met.

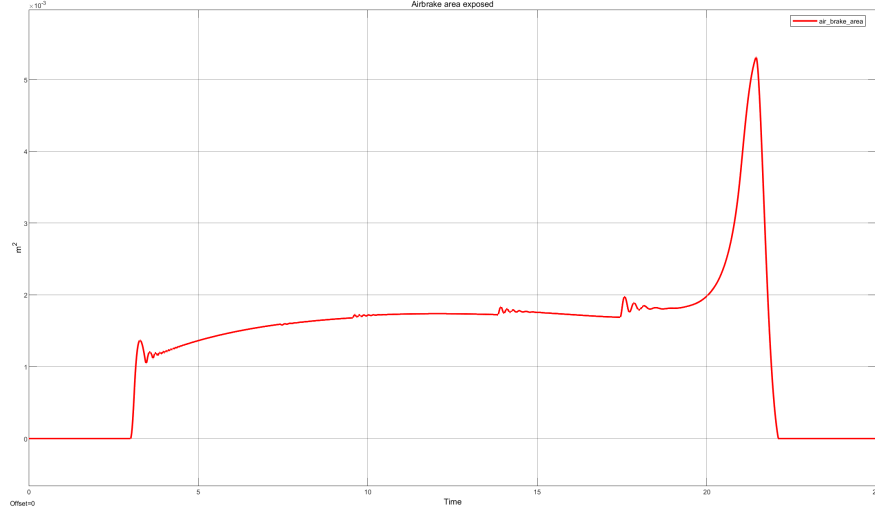


Figure 15: Airbrake area exposed targeting 2700m

In the altitude graphs with control authority the target setpoint selected by the user is met. Hitting exactly 2700m where the control makes optimal decisions and actuates based on the next instantaneous velocity required.

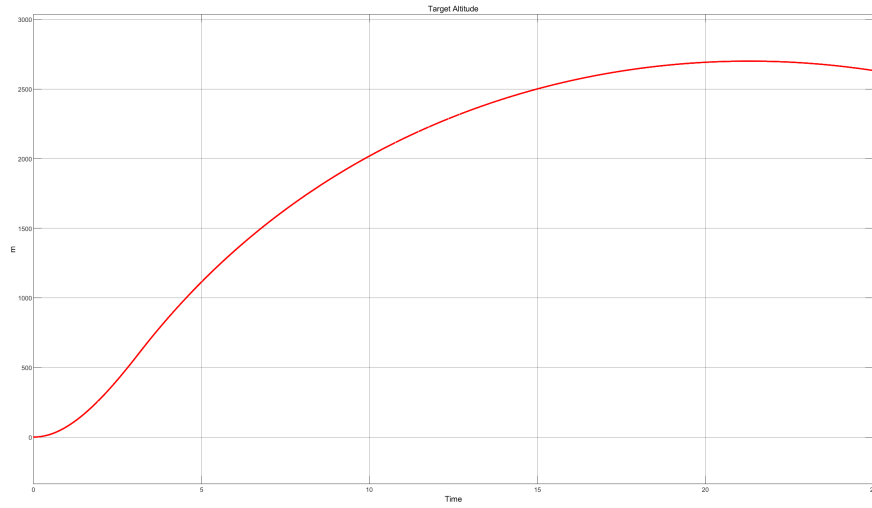


Figure 16: Target Altitude of 2700m

Disturbance and noise analysis Sensor noise was then added to the system, replicated from the data sheet. It was then added to the controller to test if it can counteract the noisy velocity measurements. It can be seen below that the system is robust enough where it will still provide the next instantaneous velocity for the controller. The airbrakes area exposed can be seen counteracting the noise where the target setpoint is still met where the altitude hits around 2700m.

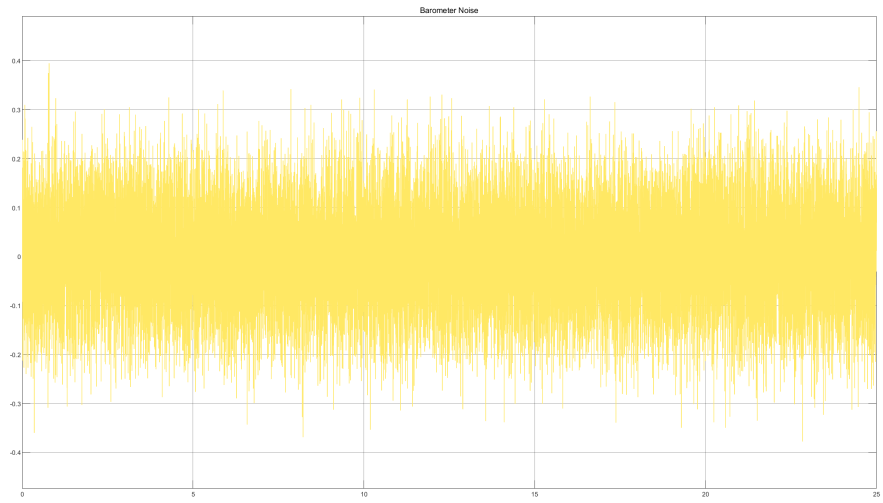


Figure 17: Barometer noise

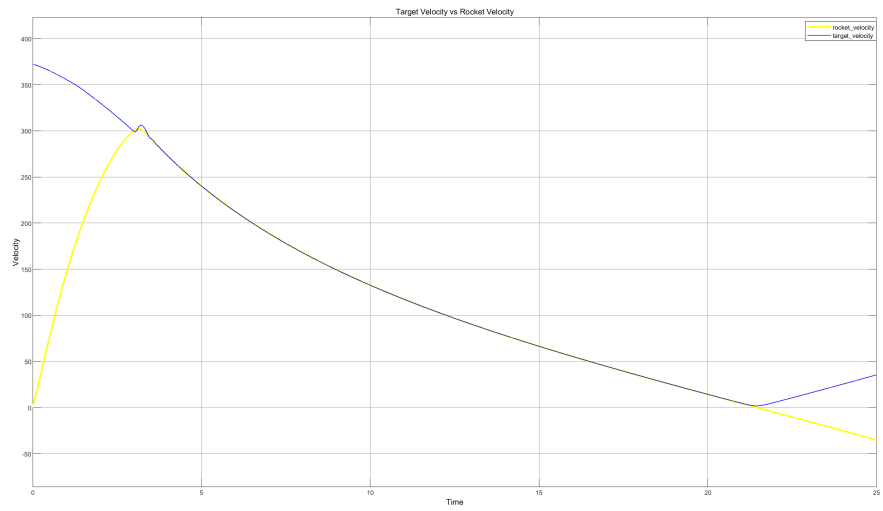


Figure 18: Trajectory planner with disturbance

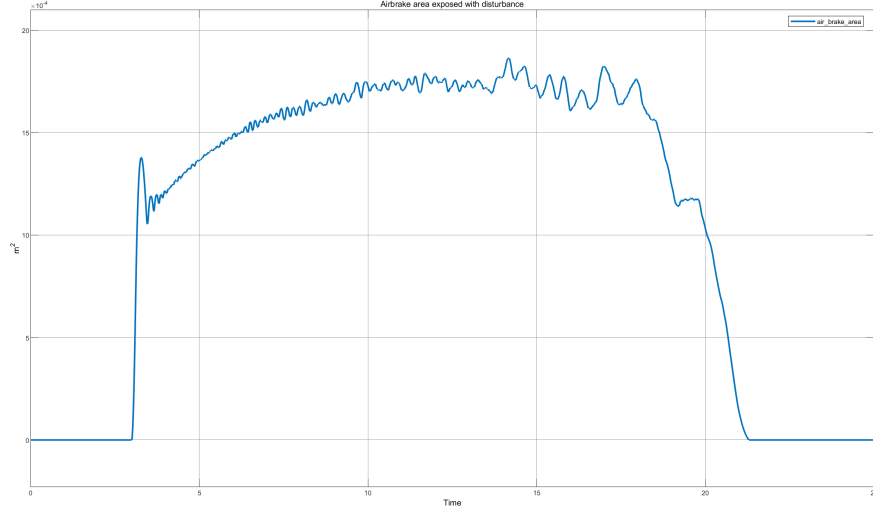


Figure 19: Airbrake area exposed with disturbance

Extra feature: An extra feature that was added was the apogee changing model where in case the motor does underperform or overperform the controller can still be used. For example, if the motor underperforms then it won't reach the desired altitude set by the user and if it overperforms the airbrakes might not have enough surface area slow down the rocket to its desired altitude effectively oversaturating the controller. Therefore, an algorithm was implemented where based on the apogee prediction model it will predict ahead on where the height will be, provided this information if the motor has complications we will have a 5-10 percent tolerance range. Meaning if it falls within that tolerance range the rocket can still hit the desired altitude and its controllable using the airbrakes. If it isn't then it will change it to a new apogee that can be controllable based on the apogee prediction model.

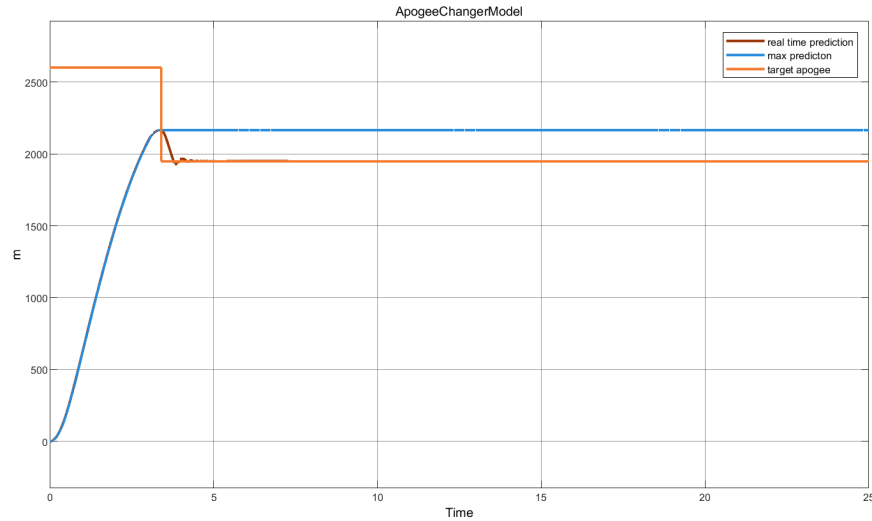


Figure 20: Desired altitude changed with 80percent motor performance

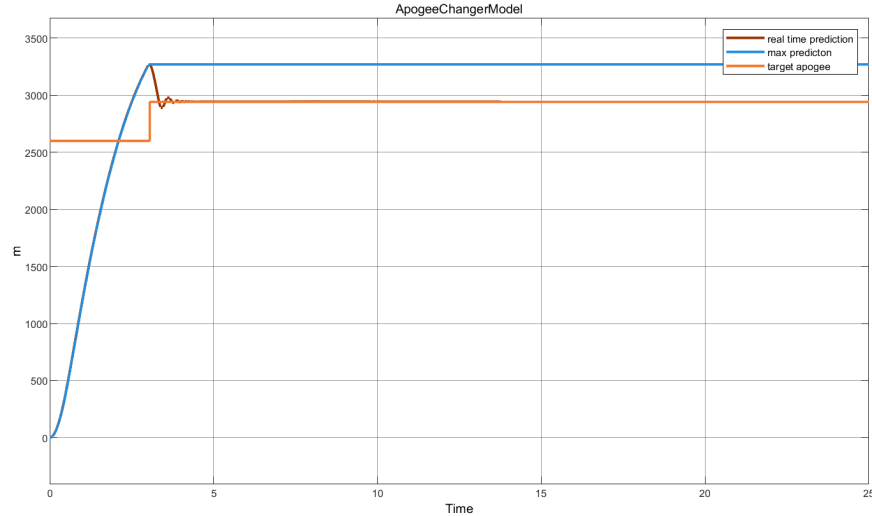


Figure 21: Desired altitude changed with 120percent motor performance

14 Process in the loop validation

Once the simulations have been validated and converted from blocks in the system into code (C/C++), checking for any bugs in the code is crucial as it can allow people to run the actual controller from the microcontroller that will be used when flashed into the rocket. A few important checklists when running the test is finding out the real-time performance, validation of the code, early detection of bugs, and improving overall system stability.

Real-Time Performance

- To validate the simulation, the controller code will need to be first tested on the actual processor. This would allow us to examine if the controller can handle the computations and to see how fast the controller rate, analyzing what could happen during real-life scenarios.

Validation of Code

- What Processor-in-the-Loop (PIL) does is it provides a bridge between testing the software in the loop while also testing the hardware. This makes sure that the code that has been created from Simulink doesn't have any clashes and functions accordingly in the microprocessor.

Early Detection of Bugs

- PIL is crucial when testing since it communicates from the hardware side of things to the simulation environment. This would allow you to see if all the code blocks and functions that have been made are working as intended; if not, early detections can be seen, allowing you to correct these beforehand.

Improving Overall System Stability

- By implementing the controller algorithm on the processor, it ensures and validates that the system is responding as it should while also mimicking real-world environments. This would make sure and provide confidence that the code implemented is fully reliable and uploaded on the final iteration.

14.0.1 PIL Results

Courtesy of Professor Ricardo's PIL simulation on cruise control, this was updated to suit the needs of confirming the validity of our controller. The PIL simulation was done through simulink, which uses a serial

communication block to communicate between the Arduino board and the simulation. The microcontroller used was the teensy 4.0, which cross-compile with the target hardware; the controller implementation is inside the hardware where incoming bytes are sent from the simulation into the Arduino code and then back into Simulink to capture the data. The hardware chosen to cross compile was the teensy 4.0 where the right com port was needed in order to transfer the data, also since the arduino has a -10 to 10v with 0.001 resolution, gains were picked carefully in order to scale the data coming in and out.

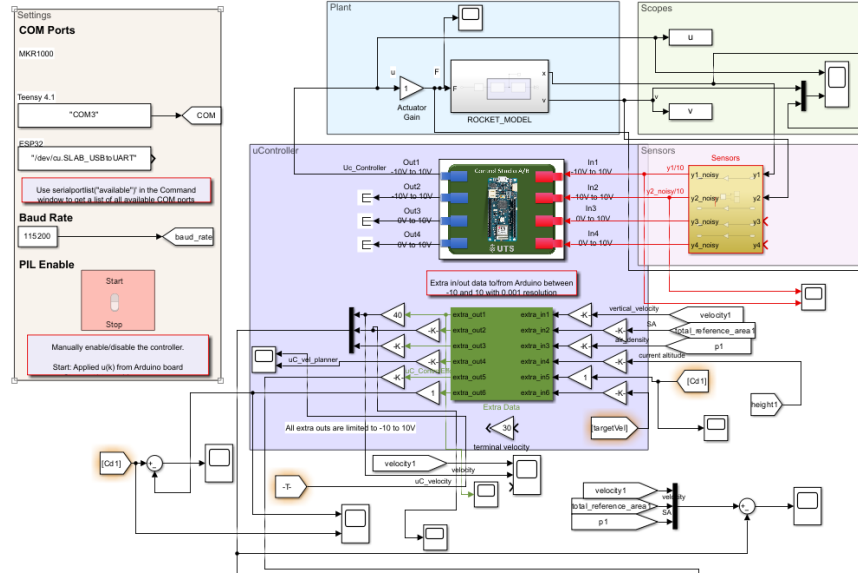


Figure 22: Processor in the loop

Shown in the graph below it can be seen that the microcontroller control matches with the simulation data. Confirming that the controller will work in a real flight scenario inside the hardware chosen (teensy 4.0). The target altitude that was set was 3000m also confirming the controller inside the hardware does its job slowing down the rocket to the desired setpoint. Through confirming with processor in the loop provides confidence in simulation and also flashing it into the hardware proving its flight ready.

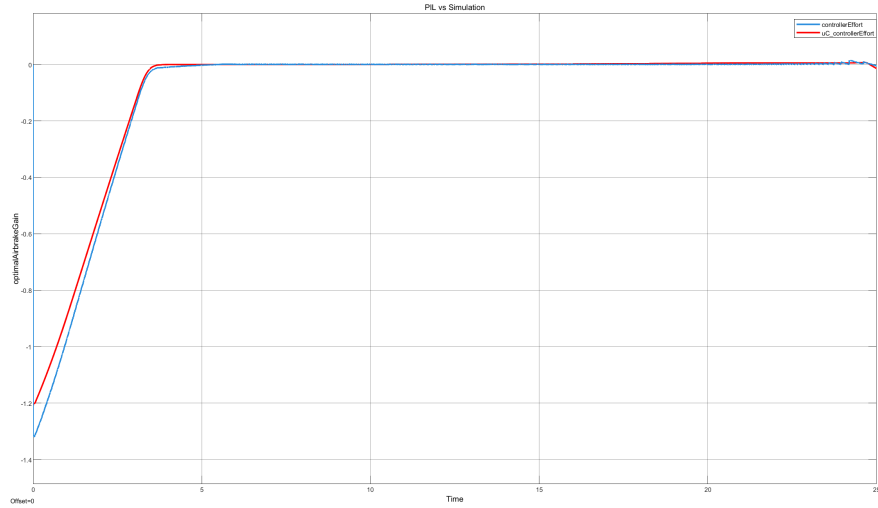


Figure 23: Processor in the loop validating the controller

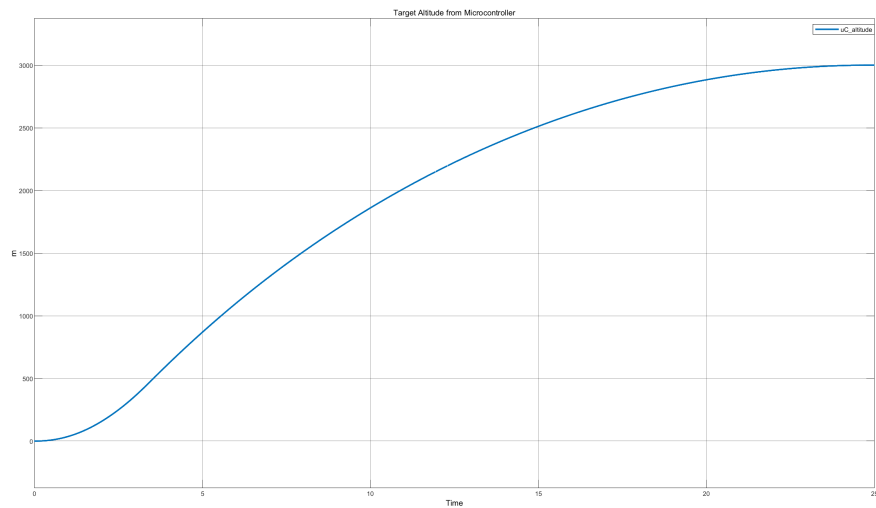


Figure 24: PIL with target altitude of 3000m

15 Post flight analysis

15.0.1 Open loop test

The first launch objective was to grab data on how much impact the airbrakes had on the system. This was the open loop test where possible system identification can be used after analyzing the airbrake characteristics. This would allow us to confirm if the impact of the airbrakes match with the simulation. Having the airbrakes actuate later in the stage would mean the airbrake response would be minimal since, at lower velocities, it would experience minimal drag force. Therefore the code was set in a way where it will actuate a second after motor burnout during the coasting stage; this would allow us to see a system response change with the airbrakes as it will affect the acceleration of the rocket the most experiencing higher velocities.

From the open loop test, the rocket landed successfully, with the data successfully retrieved from the avionics board. Runcams have also been successfully retrieved, showing evidence of the actuation of the airbrakes during its flight duration. The acceleration data evidently changes during its actuation where you could see the gradient change of the acceleration curve decrease to roughly 0.5 m/s^2 . To confirm this the same flight conditions were also used in Simulink to mimic what happened during the launch and can also be seen to decelerate roughly 0.6 m/s^2 . Unfortunately, due to a leak in the motor, there was a severe case of underperformance where the altitude of the rocket went up roughly 700m. Either way, this still confirmed the validity of the airbrake's performance, allowing us to further tune the controller for closed loop, confirming that the airbrakes simulation matches with the real-life data retrieved. Due to the drag force equation, the reason why the acceleration starts increasing again is due to the velocity decreasing, leading to the drag of the airbrakes being less effective since initially, when the airbrakes are opened, the velocity is high.

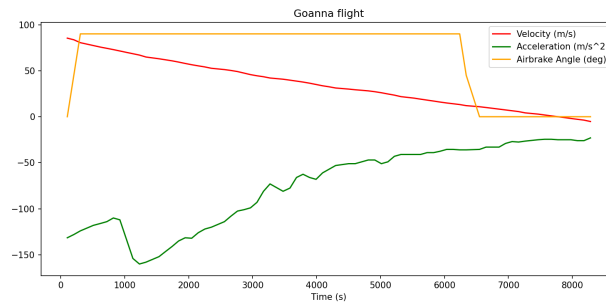


Figure 25: Post flight data of airbrakes performance

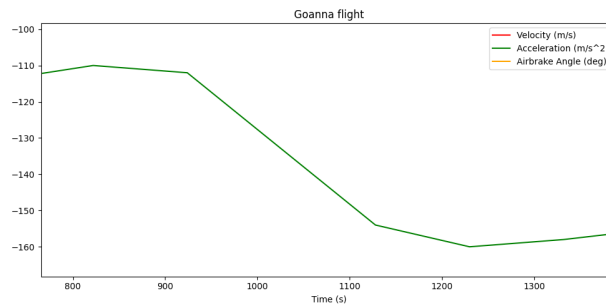


Figure 26: Zoomed in analysis of the decrease in acceleration

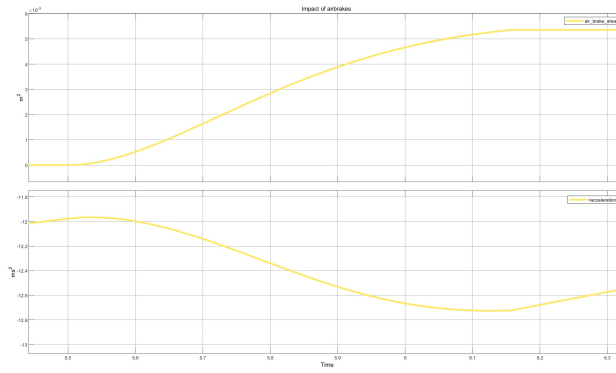


Figure 27: Simulation of airbrakes performance on the day

15.0.2 Closed loop test

All the objectives have been met and everything was prepared for the launch. Unfortunately, the flight computer reset on the actual pad, meaning the parachutes never deployed, leading to the rocket going ballistic. The SD card was chipped, and so was the runcam leading to no confirmation of the controller actuating the airbrakes after the coasting stage. But through PIL validation closed loop behaviour was behaving normally.

16 Findings and discussions

Overall the project is successful where almost all the objectives have been met and satisfied. From the simulation perspective, the controller seemed to work flawlessly, reaching the desired altitude. From modelling the air-brakes to simulation it can be seen that the airbrakes can slow down the rocket at a desired altitude within the controllable region. After validating with the simulation the next step was to validate the controller in the hardware and it can be seen that the control effort matches with that of simulation sampling at a rate of 100hz. This was then flashed into the hardware where first the open loop test was done in order to validate the impact of the airbrakes against drag with simulation. This would allow us to trust the simulation a bit more confirming that when actuated matches the change in acceleration in simulink, from the graphs shown it can be seen that the rocket is slowed down at about 0.6 m/s^2 , matching with the simulation data mimicking the flight on the day. Unfortunately due to a motor leak we couldn't get the velocities we wanted but still able to find validation with simulation. In the closed loop test unfortunately the flight computer reset leading the rocket to go ballistic where the data was corrupted and broken.

17 Conclusion

Overall it can be seen that most if not all the objectives were complete. From the ideation phase of creating morph tables was done through spitting out different ideas on what the avionics/PCB should look like with the airbrakes. Servo, microcontroller and sensors were chosen carefully. Afterwards simulations were done and validated where it uses different sub systems like the drag coefficient estimator, velocity planner and controller where they all have to work accurately in order to reach a desired height. Processor in the loop was also done and validated with simulation allowing us to be confident with having the final code into the hardware. I was also able to get flight data on the day with footage of the airbrakes working and having accurate results compared to simulink. This project provides details of the importance of gain scheduling optimising the controller for different operating points. This would allow the user to characterise the rocket for different velocity regions without the risk of running only one controller gain for the entire flight. This provides the UTS rocketry team a foundation implementing control in an airbrake system where this can be used for future tests and competitions.

18 Acknowledgments

Thank you to all the members of UTS Rocketry who worked extremely hard manufacturing the rocket allowing the testing of the airbrakes, Ricardo and Ignacio, for sharing their insightful knowledge and providing valuable feedback.

19 Appendices

1	Control Simulation						
1.1	Update and check the non linear model of the rocket	[Name]	Thu 2/29/24	Mon 3/04/24	5	100%	3
1.2	Derive the linear model equations of the rocket		Tue 3/05/24	Sat 3/09/24	5	100%	4
1.3	Speak with mechanical team on what airbrakes are viable		Mon 3/11/24	Thu 3/14/24	4	100%	4
1.4	Finalise Airbrakes		Wed 3/20/24	Sat 3/23/24	4	100%	3
1.5	Check if 3D printed airbrake tolerances are right		Mon 3/25/24	Sat 3/30/24	6	100%	5
1.6	Print final airbrake design		Mon 4/01/24	Mon 4/08/24	8	100%	6
1.7	Pick microcontroller that is being used		Thu 4/11/24	Mon 4/15/24	5	100%	3
1.8	Pick servo design		Sat 4/13/24	Wed 4/17/24	5	100%	3
1.9	Print final PCB design		Sat 4/20/24	Mon 4/29/24	10	100%	6

Figure 28: Timeline of project

2	Code the firware for the open loop airbrakes test with all the sensors	Mon 4/29/24	Wed 5/08/24	10	100%	8
2.1	Do a open loop actuation test with firmware	Fri 5/10/24	Sun 5/19/24	10	100%	6
2.2	Linearising the trajectory of the rocket across mulitple operating points where the airbrake would affect the apogee the most	Wed 5/15/24	Wed 5/22/24	8	100%	6
2.3	Rederive the LQR controller matrices to include another variable which is angle	Thu 2/01/24	Sun 2/04/24	4	100%	2
	Update the LQR controller to be adaptive where it can choose its own optimal solution every instance in the trajectory of the rocket	Thu 5/30/24	Thu 6/06/24	8	100%	6
0.0.1	Update the LQR controller to be adaptive where it can choose its own optimal solution every instance in the trajectory of the rocket	Fri 6/07/24	Fri 6/14/24	8	100%	6
0.0.2	Test servo actuation	Fri 6/07/24	Tue 6/11/24	5	100%	3
0.1	Once the model has been verified, the controller blocks need to be converted into code for SIL testing	Sat 6/15/24	Mon 6/24/24	10	100%	6

Figure 29: Timeline of project

0.2	PIL testing with the Teensy 4.1 to verify and observe the microcontroller performance with the simulations	Tue 6/25/24	Sat 7/06/24	12	100%	9
1	Pre-Flight Testing					
1.1	Airbrake calibration to find a relationship with servo angle and airbrake surface area					
1.2	{Sub task} Calibrate the encoder to ensure feedback. Where this is a quadrature encoder where it measures the tick rate, this needs to be converted into angle to know how much the servo has actuated					
1.3	Hardware testing to confirm the behaviour of airbrakes					
1	Flight Testing (April, August, September Mullaley Launch)					
1.1	Flash the the airbrakes code onto the microcontroller					
1.2	Actuation test to make sure the controller is functioning as intended					
1.3	Vacuum pressure jar test					

Figure 30: Timeline of project

1	Flight Testing (April, August, September Mullaley Launch)					
1.1	Flash the the airbrakes code onto the microcontroller					
1.2	Actuation test to make sure the controller is functioning as intended					
1.3	Vacuum pressure jar test					
1	Post Flight Testing					
1.1	Collect and analyse the data given					
1.2	Compare if the controller reached the optimal height					

Figure 31: Timeline of project

Apogee Control System					
Function/Feature		Idea 1	Idea 2	Idea 3	Idea 4
Algorithms	Solution	LQR (2023 revision)	MPC	ID	Nonlinear MPC
	Pro	Already experienced	Factors in additional	Maybe easier to get a	Will provide more accurate results
	Con	Doesn't have the ability to	More computation on	Doesn't account for future	Harder to implement due to its
State estimation	Solution	Extended Kalman Filter	Moving Average filter	Lowpass filter	Complementary Filter
	Pro	Experience using it in SAC	Relatively simple method of	Relatively simple method of	Relatively easier to implement
	Con	Not as accurate as Kalman	Low filter resolution lag	Low resolution signal	Simple filter, less accurate as it
Actuate actuators	Solution	Linear Actuator	Stepper motor	Brushless DC	
	Pro	Has previous experience using in SAC	Very accurate control and	Cheap	Very accurate
	Con	Susceptible to damage due to	Butt	Open loop, meaning no	Expensive, requires extra
Microcontroller	Solution	Teensy 4.0	ESP32	Teensy 4.1	
	Pro	Experience using it in SAC	Experience using it in SAC	Unlike the teensy 4.0 it	Doesn't have extra USB
	Con	Boards already in inventory	2023 11 10 (Right computer,	device, it has a 500MHz	speed
Actuate actuator power management	Solution	Back Converter	Voltage Regulator	Boost converter	
	Pro	Had experience with it	Easy to implement	Unidirectional, stable output,	
	Con	Expensive and can get	More expensive than some	comparable, less efficient	
Actuate shape	Solution	Can easily fit 3 into 1	Can easily fit 3 into 1	Can easily fit 3 into 1	
	Pro	Can easily fit 3 into 1	Can easily fit 3 into 1	Can easily fit 3 into 1	
	Con	Can easily fit 3 into 1	Can easily fit 3 into 1	Can easily fit 3 into 1	
Actuate paddle material	Solution	SS-304	Aluminum	Stainless Steel	
	Pro	Has experience with it	Has experience with it	Has experience with it	
	Con	Has experience with it	Has experience with it	Has experience with it	
System Model	Solution	IDOF rocket under gravity	IDOF rocket under gravity	IDOF rocket under gravity	IDOF rocket, accounting for
	Pro	Much simpler model and	More accurate and also	Describe the behaviour of	Modelling rocket angle of attack
	Con	Not as accurate as	More accurate and also	More accurate and also	More accurate and also
State Indication	Solution	Booster	Long lifespan	Long lifespan	
	Pro	Low cost, robust	Long lifespan	Long lifespan	
	Con	Limitations in terms of	Long lifespan	Long lifespan	
Actuate mechanism	Solution	Easy to design	Easy to design	Easy to design	
	Pro	Easy to design	Easy to design	Easy to design	
	Con	Easy to design	Easy to design	Easy to design	

Figure 32: Ideation phase using morph tables for component selection

Actuate mechanism	Linkages	Gears	Wheel	Springs	Lead screw (ball)
	Easy to design	Easy to design	Easy to design	Easy to design	Easy to design
	More than 5 years which may increase complexity	More than 5 years which may increase complexity	More than 5 years which may increase complexity	More than 5 years which may increase complexity	More than 5 years which may increase complexity
Next stage of flight for actuator to deploy	During launch ascent	During launch ascent	During launch ascent	During launch ascent	During launch ascent
	Easier to simulate during the	Easier to simulate during the	Easier to simulate during the	Easier to simulate during the	Easier to simulate during the
	More than 5 years which may increase complexity	More than 5 years which may increase complexity	More than 5 years which may increase complexity	More than 5 years which may increase complexity	More than 5 years which may increase complexity
Control design simulation	MATLAB	Free	Free	Free	Free
	Has experience with it	Has experience with it	Has experience with it	Has experience with it	Has experience with it
	Can be done in a	Can be done in a	Can be done in a	Can be done in a	Can be done in a
Rocket flight simulation	CFD	CFD	CFD	CFD	CFD
	Can be done in a	Can be done in a	Can be done in a	Can be done in a	Can be done in a
	Can be done in a	Can be done in a	Can be done in a	Can be done in a	Can be done in a
Actuation feedback encoder	Absolute rotary encoder	Incremental rotary encoder	Incremental rotary encoder	Incremental rotary encoder	Incremental rotary encoder
	Absolute rotary encoder	Incremental rotary encoder	Incremental rotary encoder	Incremental rotary encoder	Incremental rotary encoder
	Absolute rotary encoder	Incremental rotary encoder	Incremental rotary encoder	Incremental rotary encoder	Incremental rotary encoder

Figure 33: Ideation phase using morph tables for component selection

Trajectory planner model

$$\begin{aligned}
 m \cdot \frac{dV}{dt} &= -\frac{1}{2} \rho C_d A V^2 \\
 \frac{dV}{dt} &= -\frac{1}{2m} \rho C_d A V^2 \\
 V_t &= \sqrt{\frac{2mg}{\rho C_d A}} \\
 x &= \int V dt \\
 x &= \frac{m}{\rho C_d A} \ln \left(g - \frac{1}{2} \rho C_d A V_i^2 \right) + x_i
 \end{aligned}$$

$$x_f = x_i + \frac{m}{PC_d A} \ln \left(\frac{g}{g + \frac{1}{2} PC_d A V_i^2} \right)$$

$$x = \frac{m}{PC_d A} \ln \left(\frac{g}{g + \frac{1}{2} PC_d A V_i^2} \right) + x_i$$

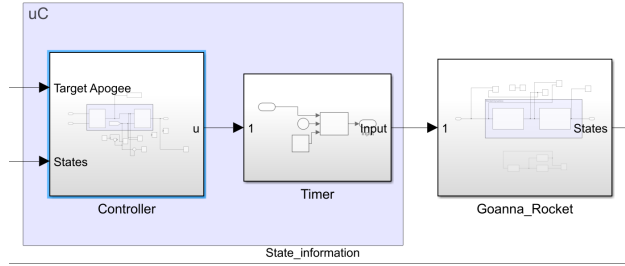


Figure 34: High-level architecture

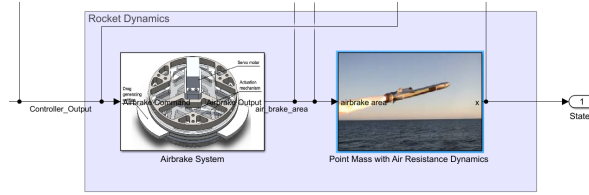


Figure 35: High-level architecture with airbrakes and rocket model

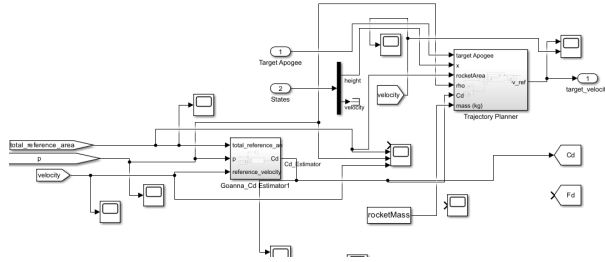


Figure 36: Velocity tracker inside with drag coefficient estimator

```

function Cd = cd_estimator(area, rho, velocity)
% coefficients = [intercept, Area, Density, Velocity, Area^2, Area*Density, Area*Velocity, Density^2, Velocity^2];
coefficients = [1.271359304570330, -89.2957223507969, -0.139915015099874, -0.000642306508177183, 2402.82168908320, 5.06245068661195,
% Manually generate polynomial features for scalar inputs
X_pred_poly = [1, ... % Intercept term
area, ... % Area
rho, ... % Density
velocity, ... % Velocity
area^2, ... % Area^2
area * rho, ... % Area * Density
area * velocity, ... % Area * Velocity
rho^2, ... % Density^2
velocity^2]; % Velocity^2

Cd = X_pred_poly * coefficients';
end

```

Figure 37: Inside CFD estimator function

```

function u_k = fcn(Vref, x)
    %% Measurements

    xref_k = Vref;
    x_k = x; % measured system states
    u_k = 0; % Initialize u_k

    if x <= 40
        K = -0.557604560965188;
    elseif x <= 80
        K = -0.149050395144053;
    elseif x <= 140
        K = -0.050007237207140;
    elseif x <= 180
        K = -0.021944352703524;
    else
        K = -0.035; % Default value for K
    end

    % Calculate control signal
    u_k = -K * (x_k - xref_k);

    % Ensure u_k is non-negative
    if u_k < 0
        u_k = 0;
    end

    return
end

```

Figure 38: Inside gain block

```

function K = GainSchedulingPolynomialAlgorithm(x)
    %#codegen
    %k = Gain
    %X = rocket velocity

    % Define the coefficients
    a = -1e-5;
    b = 0.0037;
    c = -0.3743;

    % Compute the quadratic equation
    K = a*x^2 + b*x + c;

```

Figure 39: Polynomial lookup table

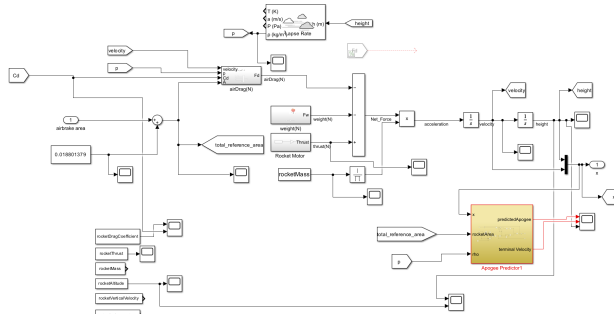


Figure 40: Rocket model

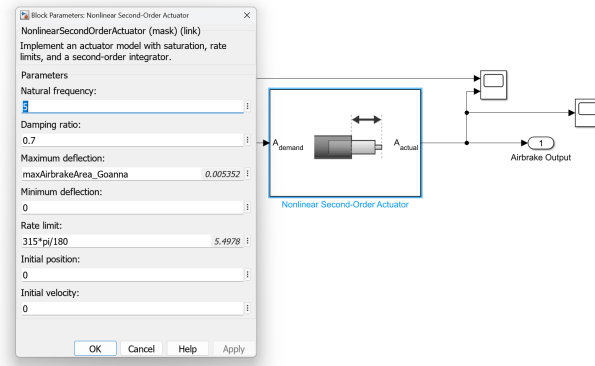


Figure 41: Saturator

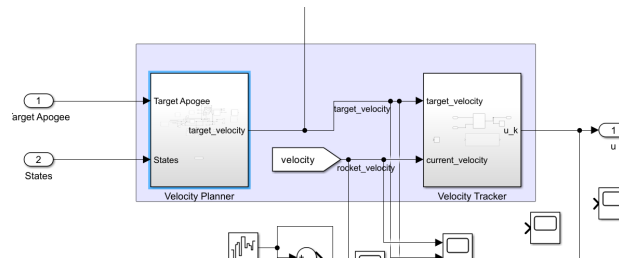


Figure 42: Velocity tracker planner

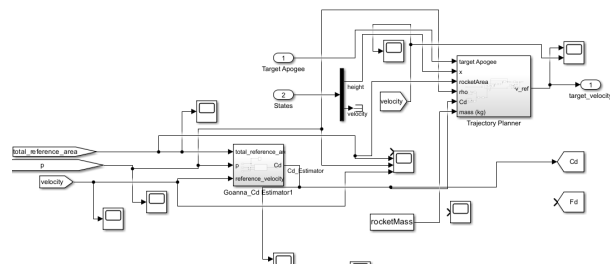


Figure 43: velocity tracker with CD estimator

Appendix I

Project Title:	Airbrakes Rocketry		
Student Name:	Mason Ma	Supervisor Name:	Ricardo Aguilera Echeverria
Date	Event	Topic of Communication	Outcome
24/3/2024	Online Meeting	Went through the research question	Discussed about my thesis direction with the question
1/4/2024	In person	Discussed the project timeline	Had a look at Gantt chart and was satisfied
10/5/2024	In person	Review of draft research proposal	Advised me to improve my introduction and explain the research question more

Figure 44: Communication Log

References

- Atman, M. W. S., & Bambang, R. T. (2015). Design and implementation of gain-scheduling flight control system on pc/104 platform. *IEEE Xplore*. <https://doi.org/10.7352504>
- Bregin, E., & Khalid, M. (2024). The development of a drag-modulating closed-loop feedback system to control a rocket's ascent. *AIAA*. <https://doi.org/10.2514/6.2024-2388>
- Furman, Z., Beebe, G., Bjerknes, F., & Montgomery, D. (n.d.). An autonomous air braking system for precision altitude control [National Association of Rocketry, Retrieved from <https://www.nar.org/member-only-reports/rd/NARAM-60/Furman-B-Division-Autonomous-Air-Braking-Precision-Altitude-NARAM-60.pdf>].
- Guo, Z., Yao, X., & Zhang, X. (2016). Robust gain scheduled longitudinal autopilot design for rockets during the sustaining phase. *Semantic Scholar*. <https://doi.org/10.86cdebec98ced466692f2e9b940089c7d530e383>
- Lee, H., & Alam, P. (2021). The design of carbon fibre composite origami airbrakes for endeavour's darwin i rocket. *Journal of Composites Science*, 5(6), 147–147. <https://doi.org/10.3390/jcs5060147>
- Lew, T., Lyck, F., & Müller, G. (2019). Chance-constrained optimal altitude control of a rocket. *EUCASS*. <https://doi.org/10.13009/EUCASS2019-388>
- Subramanian, S. C., Darbha, S., & Rajagopal, K. R. (2006). A diagnostic system for air brakes in commercial vehicles [September]. *IEEE Xplore*. <https://doi.org/10.1109/1688108>